

1a.

Keys: 5, 28, 19, 15, 20, 33, 12, 17

$$h(k) = k \% m$$

K is the key

$$m = 8$$

Hash table of size 8

Index	0	1	2	3	4	5	6	7
Key								

$$h(5) = 5 \% 8 = 5$$

Index	0	1	2	3	4	5	6	7
Key						5		

$$h(28) = 28 \% 8 = 4$$

Index	0	1	2	3	4	5	6	7
Key					28	5		

$$h(19) = 19 \% 8 = 3$$

Index	0	1	2	3	4	5	6	7
Key				19	28	5		

$$h(15) = 15 \% 8 = 7$$

Index	0	1	2	3	4	5	6	7
Key				19	28	5		15

$$h(20) = 20 \% 8 = 4$$

Index	0	1	2	3	4	5	6	7
Key				19	28->20	5		15

$$h(33) = 33 \% 8 = 1$$

Index	0	1	2	3	4	5	6	7
Key		33		19	28->20	5		15

$$h(12) = 12 \% 8 = 4$$

Index	0	1	2	3	4	5	6	7
Key		33		19	28->20->12	5		15

$$h(17) = 17 \% 8 = 1$$

Index	0	1	2	3	4	5	6	7
Key		33->17		19	28->20->12	5		15

1b.

10, 22, 31, 4, 15, 28, 17, 88, 59

Index	0	1	2	3	4	5	6	7	8	9	10
Key											

$H(10) = (10) \% 11 = 10$

Index	0	1	2	3	4	5	6	7	8	9	10
Key											10

$H(22) = (22) \% 11 = 0$

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22										10

$H(31) = (31) \% 11 = 9$

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22									31	10

$H(4) = (4) \bmod 11 = 4$

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22				4					31	10

$H(15) = (15) \% 11 = 4$

4 is occupied check next index, check next index of 5, 5 is available so insert there

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22				4	15				31	10

$H(28) = (28) \% 11 = 6$

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22				4	15	28			31	10

$H(17) = (17) \% 11 = 6$

6 is occupied, check next index of 7, 7 is available so insert there

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22				4	15	28	17		31	10

$H(88) = (88) \% 11 = 0$

0 is occupied, check next index of 1, 1 is available so insert there

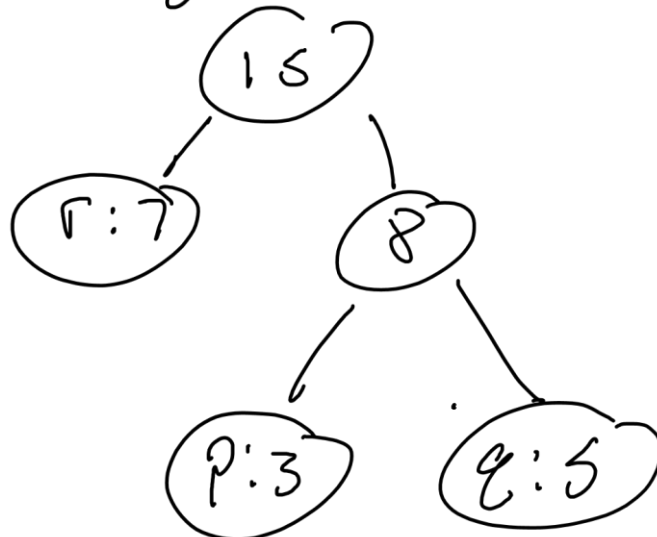
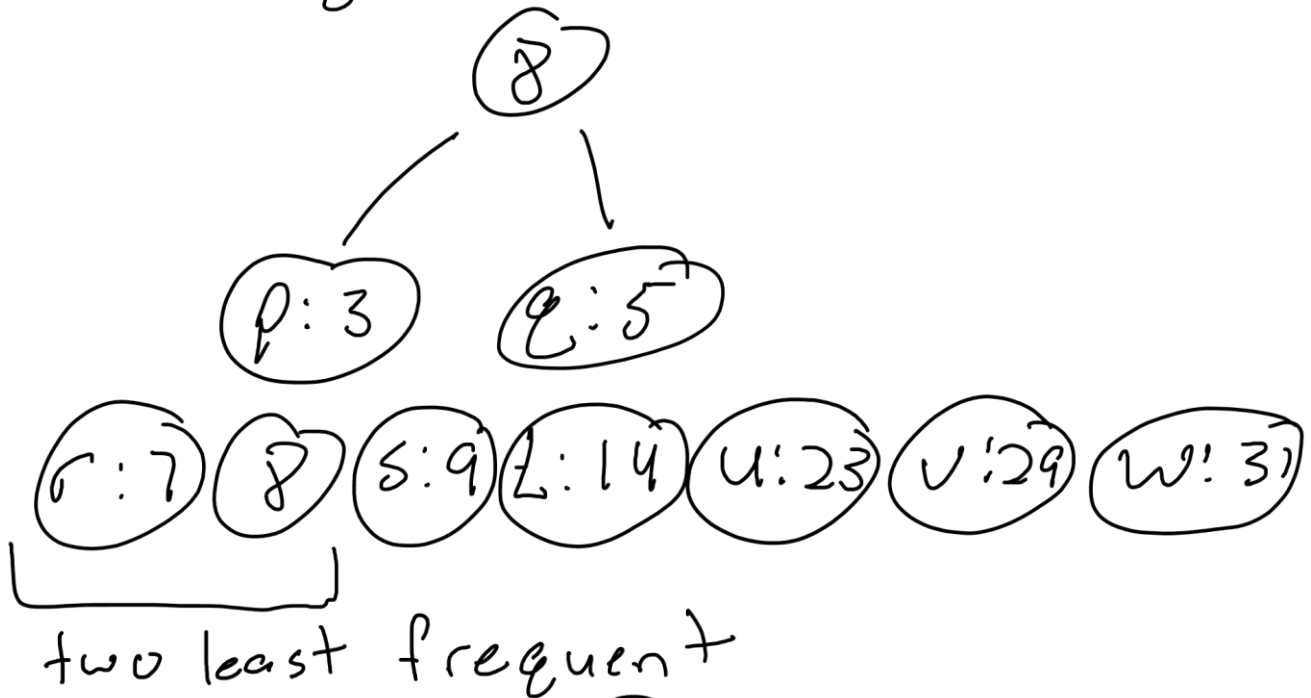
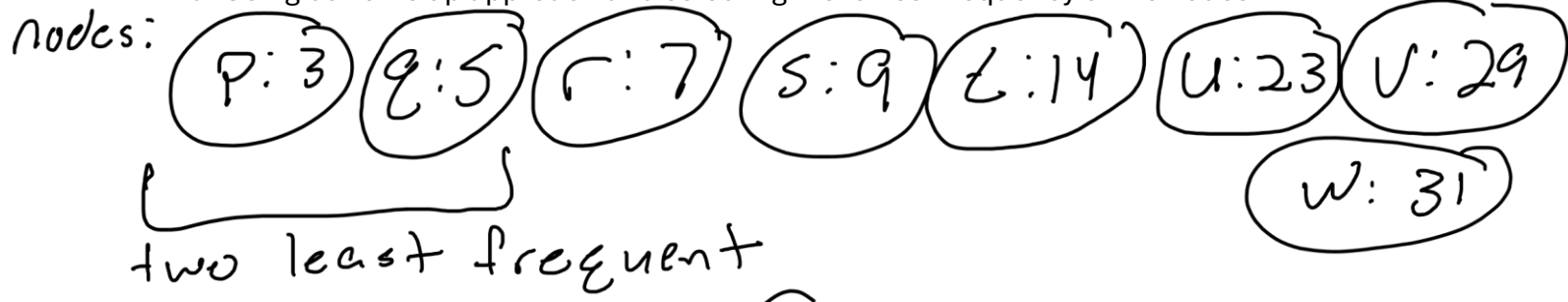
Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	88			4	15	28	17		31	10

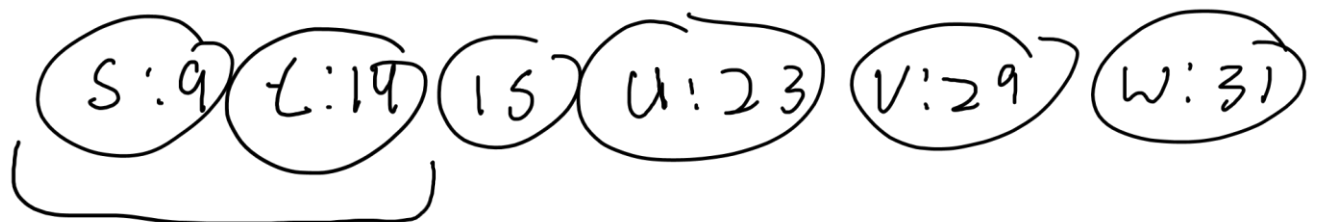
$H(59) = (59) \% 11 = 4$,

4 is occupied, check next index of 5, 5 is occupied, check next index of 6, 6 is occupied, check next index of 7, 7 is occupied, check next index of 8 is available so insert there

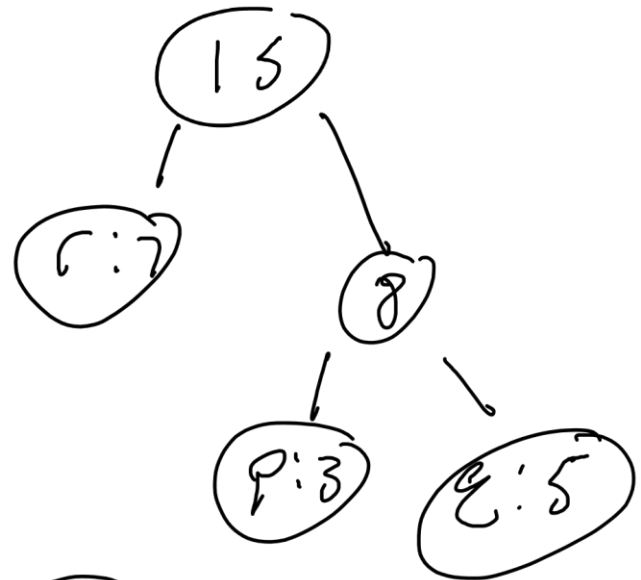
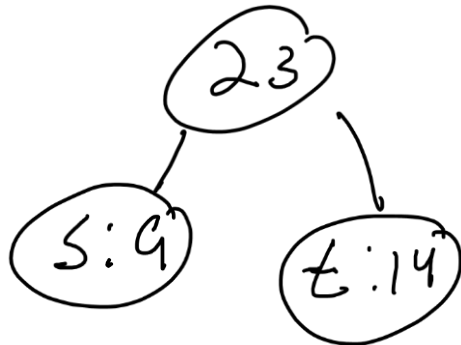
Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	88			4	15	28	17	59	31	10

2a. Using bottoms up approach and selecting the lowest frequency of the nodes

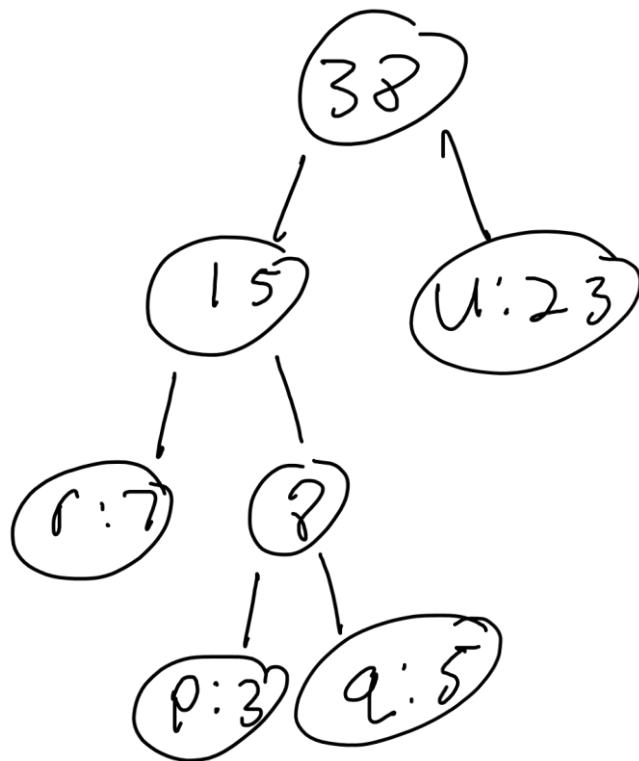
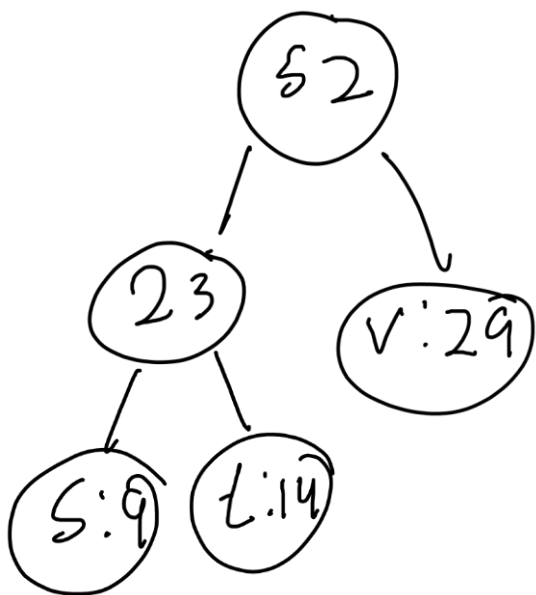
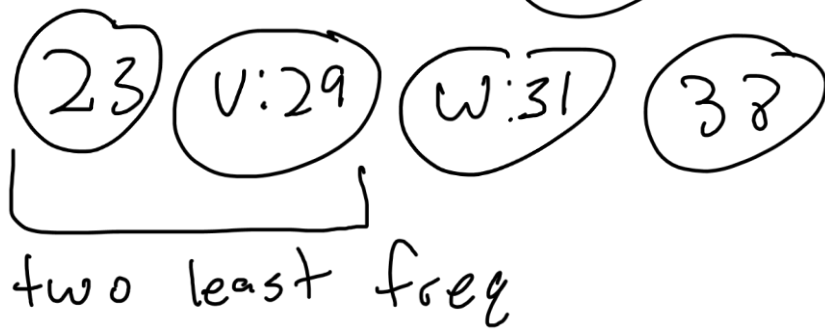
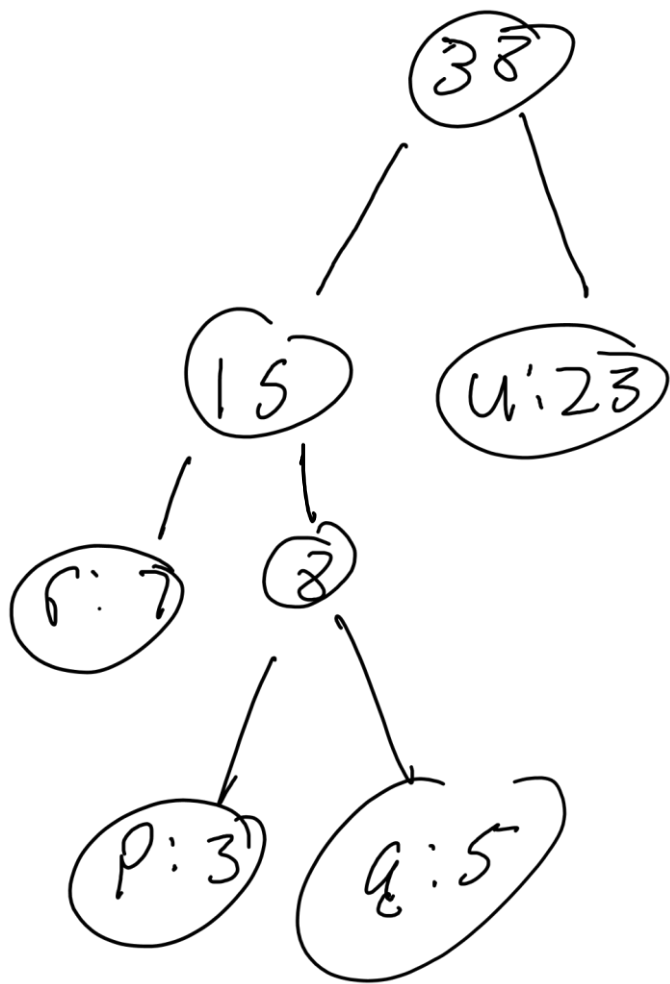
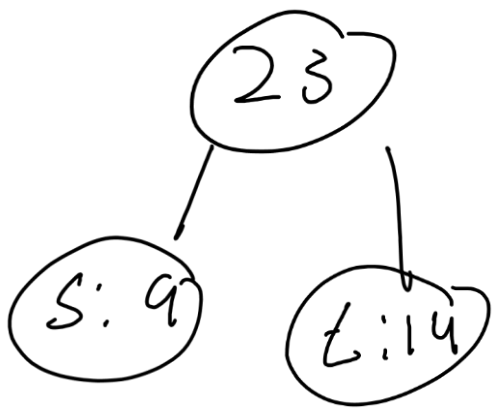


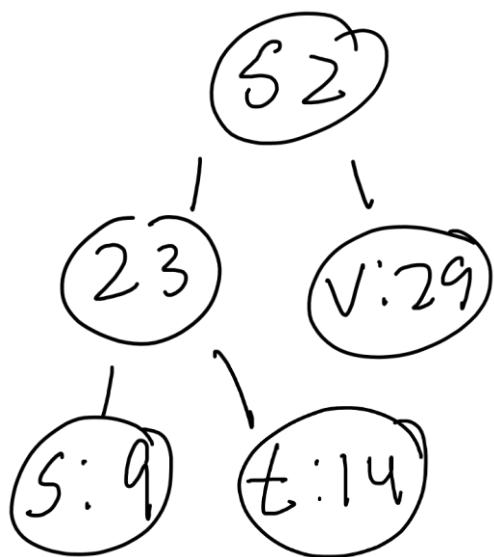


two least frequent

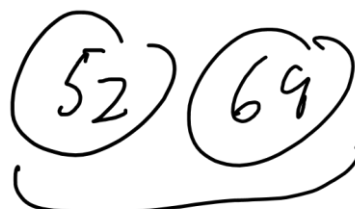
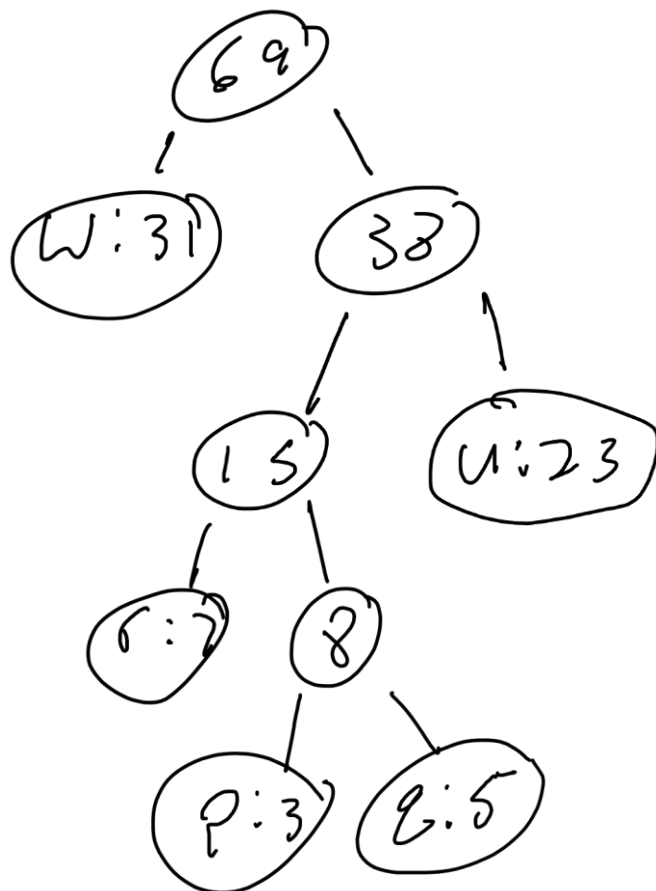


two least frequent, choosing the 23 that was their first



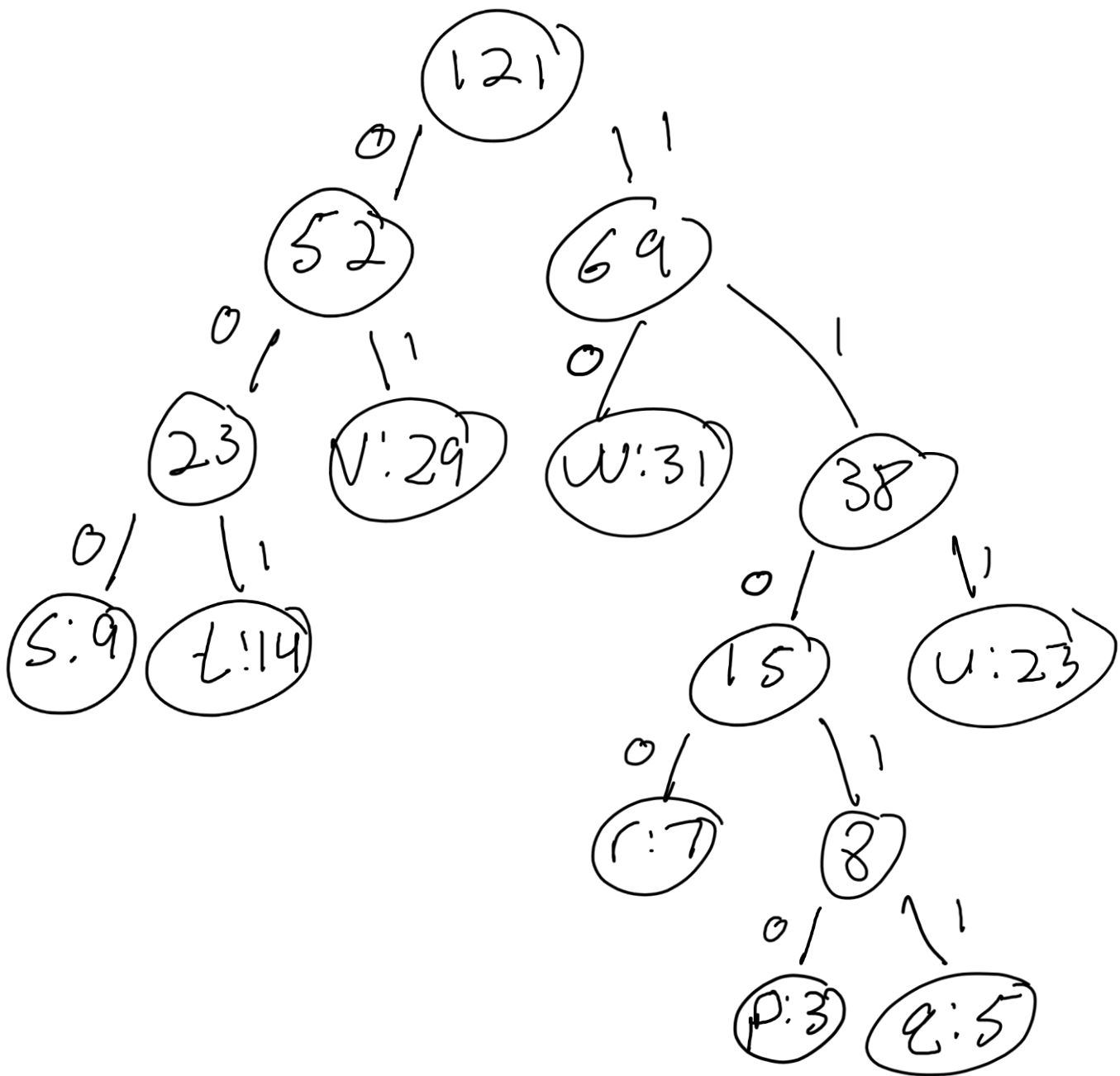


two least frequent



two least frequent

and
add 0s on left branch and 1s on right branch



2b.

P: 11010

Q:11011

R:1100

S:000

T:001

U:111

V:01

W:10

3. Pick an element then find all possibilities of elements of the rest of the array, back track and do the same. As discussed in class I followed the pseudocode from the slides.

```
import numpy as np

#return all permutations of an array of distinct numbers
# i look at this as very similar to factorial
def permutations(arr):
    # base case
    if len(arr) == 0:
        return [[]]
    if len(arr) == 1:
        return [arr]

    result = []

    def visit(cur_ans, choices):

        if not choices:
            result.append(cur_ans[:])
            return

        for num in choices:

            #add the number to cur_ans
            cur_ans.append(num)

            new_choices=[]
            for x in choices:
                if x != num:
```



```

        new_choices.append(x)

        visit(cur_ans, new_choices)

        #backtracking
        cur_ans.pop()

    visit([], arr)
    return result

print(permutations([1, 2, 3]))

```

4

- Binary Search

The screenshot shows the LeetCode interface for problem 704, 'Binary Search'. The problem description states: 'Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`. You must write an algorithm with $O(\log n)$ runtime complexity.'

Example 1:
 Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
 Output: `4`
 Explanation: 9 exists in `nums` and its index is 4

Example 2:
 Input: `nums = [-1,0,3,5,9,12]`, `target = 2`

The code editor shows a Python solution using a recursive binary search function:

```

13 def search(nums, target):
14     mid = len(nums) // 2
15
16     if nums[mid] == target:
17         return mid
18     elif nums[mid] > target:
19         return self.search(nums[:mid],
20                             target)
21     else:

```

The test results section shows 'Accepted' with a runtime of 0 ms. Two test cases are listed: 'Case 1' and 'Case 2', both marked as passed. The input for Case 1 is `nums = [-1, 0, 3, 5, 9, 12]` and `target = 9`.

- Merge Sort

← → ↻ 🔍 leetcode.com/problems/sort-an-array/

Problem List < > ⚙️

Description Editorial Solutions Submissions

912. Sort an Array

Medium Topics Companies

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`
Output: `[1,2,3,5]`
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,2,3,1]`
Output: `[1,2,3,5]`
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

7K 372 70 Online

Python Auto

```
6 def merge(left, right):
7     result = []
8     i = j = 0
9
10    while i < len(left) and j < len(right):
11        if left[i] < right[j]:
12            result.append(left[i])
13            i += 1
14        else:
```

Saved Ln 24, Col 31

Testcase Test Result

Accepted 0 ms

Case 1 Case 2

Input

`nums = [5,2,3,1]`

Stdout

- Quick Sort

← → ↻ 🔍 leetcode.com/problems/sort-an-array/

Problem List < > ⚙️

Description Editorial Solutions Subr ⌵

912. Sort an Array

Medium Topics Companies

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`
Output: `[1,2,3,5]`
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,2,3,1]`
Output: `[1,2,3,5]`
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Python Auto

```
10 # base case
11 if len(nums) <= 1:
12     return list(nums)
13
14 pivot_index = len(nums) - 1
15 pivot = nums[pivot_index]
16
17 low = 0
18 .....
```

Saved Ln 10, Col 21

Testcase Test Result

Accepted 0 ms

Case 1 Case 2

Input

`nums = [5,2,3,1]`

Stdout